

Automated SOAR Pipeline

Security Orchestration, Automation & Response

EVE-NG | FortiGate NGFW | Suricata IDS | Groq AI | n8n | Discord

Amrith K — Aspiring Network Security Engineer

July 2026

Tools Used	Detection	Automation	Enforcement	AI Triage
6	Suricata IDS	n8n Workflow	FortiGate API	Groq LLM

1. Executive Summary

This document details the design, implementation, and validation of a fully automated Security Orchestration, Automation, and Response (SOAR) pipeline built in a virtualised home lab environment. The system detects network intrusions in real time via Suricata IDS, routes enriched alerts to a Discord SOC channel with interactive response buttons, uses Groq AI (Llama 3) to triage severity, and — on analyst approval — automatically blocks the attacker IP on a FortiGate NGFW by creating address objects and updating a Quarantine_Group tied to a top-priority Deny policy. The entire response cycle, from detection to firewall enforcement, completes in under 10 seconds with zero manual policy editing.

The project was built entirely with free, open-source or eval-licensed tools and demonstrates enterprise SOC concepts including alert triage automation, SOAR orchestration, firewall API integration, and IDS signature tuning — all within a realistic multi-node EVE-NG virtualised network.

2. Lab Architecture & Network Topology

The lab was built inside EVE-NG Community Edition running as a VMware Workstation VM on a Windows host with 16 GB RAM. Five nodes were deployed across three network segments:

Node	OS / Image	IP Address	Role
FortiGate-VM	FortiOS 7.4.12	192.168.137.133 (port1) 192.168.10.1 (port2) 192.168.20.1 (port3)	NGFW / Enforcement
Attacker	Ubuntu 22.04 Server	192.168.10.10	Attack simulation (Nmap)
Victim	Ubuntu 22.04 Server	192.168.20.10	Target host (SSH / services)
Suricata IDS	Ubuntu 22.04 Server	192.168.20.20 (sniff) 192.168.137.10 (mgmt)	Inline IDS — transparent bridge

Cloud0 (pnet0)	EVE-NG bridge	192.168.137.132	Management / API access to host
----------------	---------------	-----------------	---------------------------------

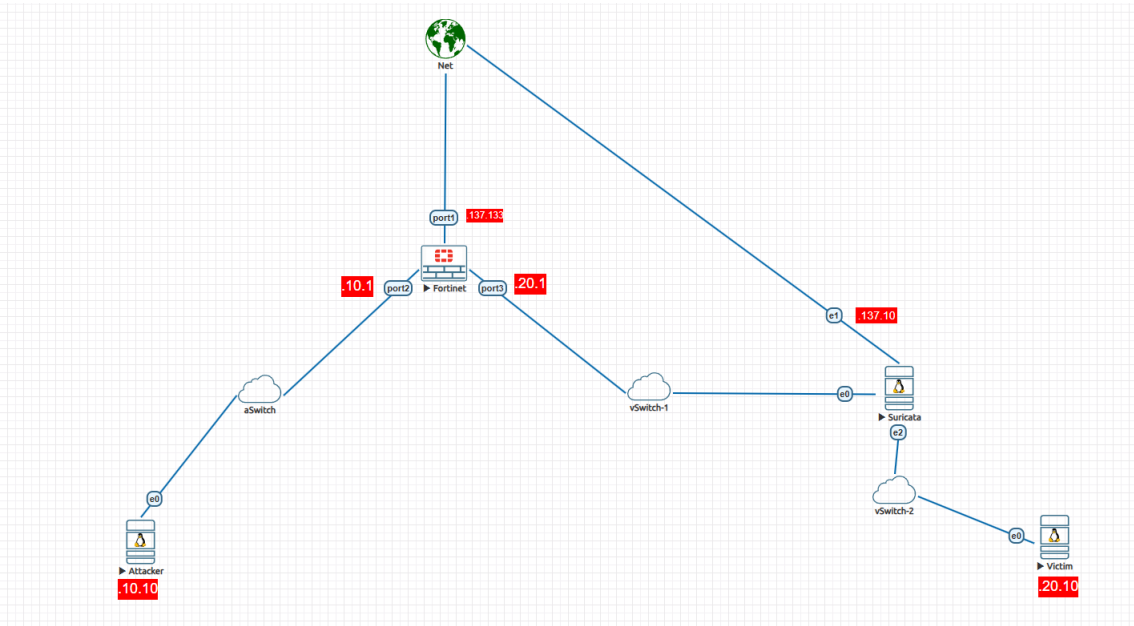


Figure 1 — EVE-NG canvas showing full topology with IP labels: Attacker (.10.10), FortiGate (port1/2/3), Suricata IDS (inline bridge), Victim (.20.10), and Cloud0 management network.

3. End-to-End Data Flow

Step 1 — Attack Simulation

Nmap aggressive scan (`nmap -A -T4`) launched from Attacker VM (192.168.10.10) targeting Victim (192.168.20.10) via FortiGate. Traffic passes through the Suricata inline bridge (ens3 → br0 → ens5).

Step 2 — Intrusion Detection

Suricata monitors all traffic on br0 in promiscuous mode. The Nmap scan triggers built-in ET signatures and custom threshold rules. Alerts are written to eve.json and simultaneously forwarded via webhook to the n8n automation server on the host.

Step 3 — AI Triage (Groq)

n8n receives the Suricata JSON alert. An HTTP Request node POSTs the alert details (src_ip, signature, severity) to the Groq API (llama-3.1-8b-instant). Groq returns a BLOCK or MONITOR decision with a one-sentence justification.

Step 4 — IF Node Routing

An n8n IF node checks whether Groq's response contains 'BLOCK'. If true, the flow continues to Discord with an interactive Block IP button. If false (MONITOR), a low-priority notification is sent and the flow ends.

Step 5 — SOC Discord Alert

A rich Discord embed is posted to the SOC channel showing: attacker IP, signature, severity, Groq's reasoning, and an interactive 'Block IP' button. The analyst reviews the alert in Discord.

Step 6 — FortiGate Enforcement

Analyst clicks 'Block IP'. Discord sends a callback to a second n8n webhook. n8n POSTs to FortiGate REST API to create address object Blocked_Attacker_192.168.10.10. A 5-second Wait node buffers the DB commit. n8n then PUTs the address object into Quarantine_Group. The top-priority SOC_Auto_Block deny policy immediately drops all traffic from the attacker IP.

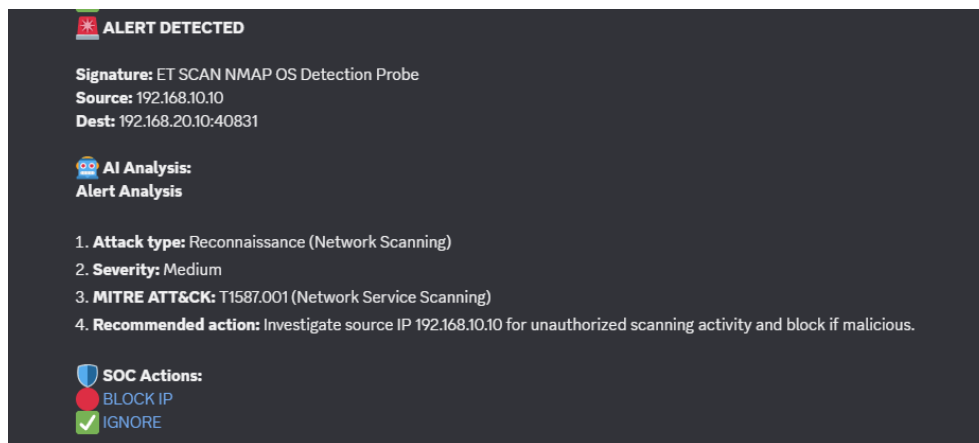


Figure 2 — Discord SOC alert showing: Signature (ET SCAN NMAP OS Detection Probe), Source IP (192.168.10.10), Groq AI analysis with MITRE ATT&CK; mapping (T1587.001), severity rating, and interactive BLOCK IP / IGNORE response buttons.

4. n8n Workflow Configuration

4.1 — Suricata Alert Triage Workflow (Groq + Discord)

This workflow receives Suricata webhook alerts, calls Groq for AI severity analysis, routes BLOCK decisions to Discord with interactive response buttons, and passes MONITOR decisions to a low-priority logging branch.

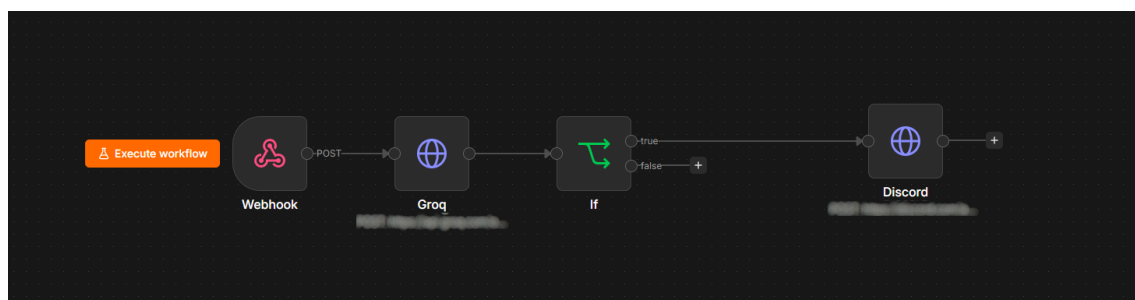


Figure 2 — n8n triage workflow: Webhook (POST) → Groq AI analysis → IF node (BLOCK/MONITOR) → Discord alert with interactive button.

4.2 — FortiGate Block Workflow (Address Object + Group Update)

This workflow is triggered by the Discord 'Block IP' button callback. It creates a FortiGate address object for the attacker IP, waits 5 seconds for DB commit, then updates the Quarantine_Group via PUT request. A critical fix was required here: the Wait node causes n8n to lose item context, so the attacker IP must be retrieved using `$('Webhook').first().json.query.ip` rather than `.item` to avoid data amnesia.

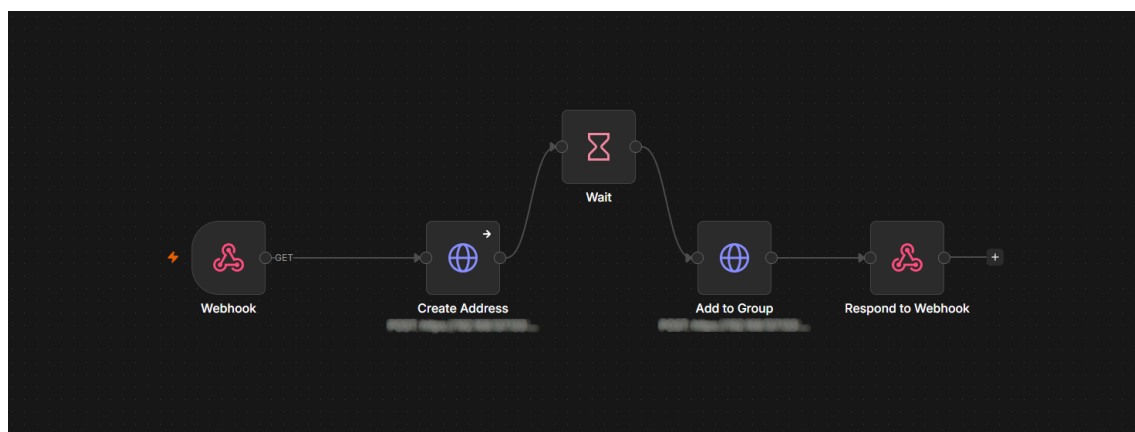


Figure 3 — FortiGate block workflow: Webhook (GET) → Create Address (POST) → Wait (5s buffer) → Add to Quarantine_Group (PUT) → Respond to Webhook.

5. FortiGate Firewall Configuration

5.1 — Policy Table & Address Group

FortiGate processes firewall policies top-down. The SOC_Auto_Block deny policy must sit at Sequence #1 above the Attacker_TO_Victim accept policy, or the accept rule will match first and the block will never take effect. The Quarantine_Group address group is dynamically populated by n8n via REST API and used as the source address in the deny policy.



Figure 4 — FortiGate GUI showing: Quarantine_Group containing Blocked_Attacker_192.168.10.10 (top), Address object list with IP/32 subnet (middle), and policy table with SOC_Auto_Block at Sequence #1 (bottom).

5.2 — Stateful Inspection Caveat

FortiGate maintains a session table and will not terminate actively established sessions when a new deny policy is applied. Traffic from a session that was already permitted will continue until the attacker stops and initiates a new connection — at which point the deny policy snaps shut immediately. This is expected FortiGate stateful behaviour and was accounted for in the pipeline design.

5.3 — REST API Configuration

A dedicated REST API admin (n8n-api) was created on FortiGate with super_admin profile and trusted host restricted to 192.168.137.1/32 (the host PC running n8n). All API calls use Bearer token authentication over HTTPS.

Key API Endpoints Used:

Method	Endpoint	Purpose
POST	/api/v2/cmdb/firewall/address	Create attacker address object
PUT	/api/v2/cmdb/firewall/addrgrp/Quarantine_Group	Add address to quarantine group
GET	/api/v2/cmdb/system/status	Health check / connectivity test

6. Proof of Concept — Before & After Block

The following screenshot demonstrates the pipeline working end-to-end. An aggressive Nmap scan (nmap -A -T4) is run from the Attacker VM before and after the automated block is applied. Before the block, the scan completes and returns open ports (SSH on 22, services on 2000 and 5060). After the block is triggered via the Discord button, the same scan returns 'Host seems down' — confirming that FortiGate is dropping all traffic from the attacker IP at the network level.



Figure 5 — Before BLOCK: `nmap -A -T4` returns open ports and OS detection. After BLOCK: same scan returns 'Host seems down / 0 hosts up' confirming FortiGate deny policy is active. Timestamps show ~1 minute between scans.

7. Suricata IDS Tuning

7.1 — Inline Bridge Deployment

Suricata was deployed as a transparent inline bridge between FortiGate port3 (vSwitch-1) and the Victim subnet (vSwitch-2). Two Ubuntu interfaces (ens3, ens5) were bridged into br0 using netplan. Suricata monitors br0 in AF_PACKET mode, seeing all traffic in both directions without being visible to endpoints.

7.2 — Alert Fatigue Mitigation

A single ICMP flood generated dozens of alerts per second, overwhelming the Discord webhook and n8n pipeline. The global threshold.config approach was unreliable due to gen_id mismatches, so thresholds were injected directly into the matching rule's .rules file:

```
threshold: type limit, track by_src, count 1, seconds 60;
```

This guarantees exactly one alert per source IP per minute, keeping the SOC Discord channel clean and the n8n pipeline stable under heavy attack traffic.

7.3 — Two-Way Street False Positive Fix

Suricata flagged both the attacker's Echo Request and the victim's Echo Reply, risking the victim's own IP being blocked. An IF node was added to n8n before the Discord webhook with condition: Source IP != 192.168.20.10. Only alerts from non-victim IPs are routed to Discord.

8. Key Challenges & Lessons Learned

• API Race Condition — Database Lock

n8n was hitting the FortiGate API to create an address object and immediately trying to add it to Quarantine_Group before the VM's disk I/O had committed the new object, causing 21-second timeouts and database lock errors. Solution: a 5-second Wait node between the two API calls eliminated all timeouts.

• n8n Context Amnesia After Wait Node

The Wait node caused n8n to lose the current item context, so `$json.query.ip` returned empty after the wait. Solution: using `$('Webhook').first().json.query.ip` forces n8n to retrieve the IP from the very start of the flow, bypassing the context loss entirely.

• FortiGate Policy Sequencing

The SOC_Auto_Block deny policy was being silently bypassed because an Accept-All policy sat above it in the table. FortiGate processes rules top-down and stops at the first match. Switching to 'By Sequence' view and moving SOC_Auto_Block to Sequence #1 immediately resolved the issue.

• Suricata gen_id Mismatch

Global threshold.config rules were silently ignored because the gen_id field in the config did not match the actual generator ID of the rule. Injecting the threshold directly into the rule's parentheses guaranteed the rate limit applied regardless of gen_id.

• pnet0 Bridge & Internet Access

EVE-NG VMs had no internet access to install Suricata because pnet0 was a bridge without IP forwarding. Enabling iptables masquerading on the EVE-NG host (POSTROUTING MASQUERADE on eth0) gave all lab VMs internet access through the NAT adapter without changing any IP addresses.

9. Tools & Technology Stack

Component	Tool / Version	Purpose	Cost
Virtualisation	EVE-NG Community	Multi-node lab environment	Free
Firewall	FortiGate-VM 7.4.12	NGFW enforcement + REST API	Eval (free)
IDS	Suricata 7.0.3	Inline intrusion detection	Free / Open Source
Automation	n8n (npm)	SOAR workflow orchestration	Free (self-hosted)
AI Triage	Groq API (Llama 3.1 8B)	Alert severity classification	Free tier
Alerting	Discord Webhooks	SOC notification + analyst interaction	Free
Hypervisor	VMware Workstation	Host for EVE-NG VM	Licensed
OS	Ubuntu 22.04 Server	Attacker, Victim, IDS nodes	Free

10. Outcomes & Skills Demonstrated

- ✓ Designed and deployed a production-quality SOAR pipeline end-to-end in a virtualised lab
- ✓ Configured FortiGate NGFW interfaces, routing, NAT policies, address groups, and REST API access
- ✓ Deployed Suricata IDS in transparent inline bridge mode with custom rule and threshold tuning
- ✓ Built multi-step n8n automation workflows with AI integration, conditional routing, and API orchestration
- ✓ Integrated Groq LLM (Llama 3) for real-time alert triage and severity classification
- ✓ Implemented Discord as a human-in-the-loop SOC notification and response channel
- ✓ Diagnosed and resolved API race conditions, firewall policy sequencing bugs, and IDS tuning issues
- ✓ Documented the full project architecture, troubleshooting steps, and lessons learned as technical reference